A Mathematical Model for Cross-Exchange Price Discovery

Eliott Mea

Kaspa Ecosystem Foundation (KEF)

DRAFT v0.0.1

Abstract

We propose a no-arbitrage framework for aggregating limit order books across multiple exchanges. By embedding heterogeneous books into a common price grid and simulating cross-venue arbitrage until overlap is exhausted, we obtain a consolidated book whose midpoint defines a fair price. To accelerate information percolation and enforce local no-arbitrage conditions, we incorporate a decentralized arbitrage network, and show that sustained price deviations require capital comparable to market-wide liquidity.

Contents

1 Aggregating Limit Order Books **2**
- 1.1 Exchanges and Filtrations . . . 2
- 1.2 Aggregation and Fair Value . . . 2

2 Decentralized Arbitrage Network (DAN) **4**
- 2.1 Motivation . . . 4
- 2.2 Architecture . . . 4
- 2.3 Arbitrage Agents and Local Aggregators . . . 5
- 2.4 Cluster Connectors . . . 5
- 2.5 Engine Nodes . . . 5

3 Robustness **6**
- 3.1 The two cracks in the wall . . . 6
- 3.2 Robustness to market manipulation . . . 7

1 Aggregating Limit Order Books

To construct a cross-exchange price oracle, we first require a unified representation of liquidity across all venues. Each exchange operates on its own native tick size and publishes updates according to its internal clock. We derive the finest price grid that is compatible with all venues, in order to simulate how the asset would be traded if it were only accessible on a single venue.

1.1 Exchanges and Filtrations

Definition 1.1. *Denote the set of exchanges $\mathcal{E} = \{1, \ldots, n\}, n \in \mathbb{N}$. Exchange $i \in \mathcal{E}$ has native tick $\eta_i \in \mathbb{Q}_{>0}$ and grid $G_i = \{k \cdot \eta_i \mid k \leq K_i\}$ where $k \in \mathbb{N}$ and $K_i > 0$. Let $D = \mathrm{lcm}(\mathrm{den}(\eta_1), \ldots, \mathrm{den}(\eta_n))$ and $d_i := D\,\eta_i \in \mathbb{N}$, set the tick-size*

$$\eta := \frac{\gcd(d_1, \ldots, d_n)}{D}$$

Then the finest common grid is given by

$$G \;= \big\{\, p_k := k\eta \mid 0 \leq k \leq K \big\}, \qquad K = \max_{i \in \mathcal{E}} K_i \tag{1}$$

Remark 1.2. *Notice that there might be elements of G that belong to no single exchange. For example, take $n = 2$ exchanges, where $\eta_1 = 4, \eta_2 = 6$. Then we have that $\eta = 2$, and thus $\{2\} \in G \setminus \bigcup_i G_i$. When working on the finest grid G, any buy or sell price that belongs to neither exchange grid G_i will be set as having volume 0 (no buy/sell orders at that price).*

Definition 1.3. *Let $m = |G|$. A snapshot of exchange $i \in \mathcal{E}$ at time $t \in \mathbb{R}_+^*$ is defined by*

$$\mathbf{x}_i^{(t)} = (\mathbf{b}_i^{(t)}, \mathbf{a}_i^{(t)}) \in \mathbb{R}_+^{2m}$$

where

$$b_{i,k}^{(t)} = \textit{total bid depth at } p_k \;\times\; \mathbf{1}\{\, p_k \in G_i \,\}$$
$$a_{i,k}^{(t)} = \textit{total ask depth at } p_k \;\times\; \mathbf{1}\{\, p_k \in G_i \,\}$$

We further define the total bid and ask profiles

$$\mathbf{B}^{(t)} = \sum_{i=1}^{n} \mathbf{b_i}^{(t)} \in \mathbb{R}^m, \quad \mathbf{A}^{(t)} = \sum_{i=1}^{n} \mathbf{a_i}^{(t)} \in \mathbb{R}^m \tag{2}$$

1.2 Aggregation and Fair Value

In order to determine the fair value of an asset at t, we take inspiration from the NA (no arbitrage) property of efficient markets. The fair value of an asset is its value if all arbitrage were filled in an instant. An algorithm that performs such an operation would simulate existing arbitrage in order to end up with the NA condition, and thus determine the fair value of the asset.

Definition 1.4. *$\forall t > 0$, define the best global bid and ask indices as*

$$k_b^{(t)} = \max\{k : \mathbf{B}_k^{(t)} > 0\}, \qquad k_a^{(t)} = \min\{k : \mathbf{A}_k^{(t)} > 0\} \tag{3}$$

Algorithm 1 Consolidated Order Book (COB) at time t

Require: $\mathbf{x}_i$ for $i = 1, \ldots, n$

Construct aggregates $\mathbf{B}, \mathbf{A}$ from (2)
Compute best indices k_b, k_a from (3)
while $k_b \geq k_a$ **do**
  $v \leftarrow \min\{\mathbf{B}_{k_b}, \mathbf{A}_{k_a}\}$
  $\mathbf{B}_{k_b} \leftarrow \mathbf{B}_{k_b} - v$
  $\mathbf{A}_{k_a} \leftarrow \mathbf{A}_{k_a} - v$
  if $\mathbf{B}_{k_b} = 0$ **then**
    $k_b \leftarrow \max\{k < k_b : \mathbf{B}_k > 0\}$
  end if
  if $\mathbf{A}_{k_a} = 0$ **then**
    $k_a \leftarrow \min\{k > k_a : \mathbf{A}_k > 0\}$
  end if
end while
Output: (k_b, k_a)

From the output of the above algorithm, we determine the fair value of an asset as

$$f_t = \frac{p_{k_b} + p_{k_a}}{2}$$

2 Decentralized Arbitrage Network (DAN)

2.1 Motivation

The following architecture arises from two straightforward questions: How do we ensure rapid aggregation of CEX data ? How do we make the market efficient ?

Given a fixed set of exchanges across the world, receiving information from all exchanges is limited by information percolation: this highlighting the importance of the location of the server running Algorithm (1). An oracle must aggregate all information in a single location. To that end, we use a weighted geometric median to approximately determine the ideal location. For now, we do not bother with precise calculations.

The second question is a bit more subtle, and requires careful analysis of information percolation in distributed systems. We leave the analysis for future work, and for now focus on the following methodology: position servers in order to close arbitrages as fast as a possible between geographically co-located exchanges. This leads to a cluster-like architecture of arbitrage algorithms.

2.2 Architecture

The Decentralized Arbitrage Network is structured around K geographical clusters $C_1, \ldots, C_K \subset \mathcal{E}$ and a small set of co-located Engine nodes. Each cluster groups exchanges such that intra-cluster communication latency remains negligible compared with inter-cluster delays. Let $\ell(i, j)$ denote the median network latency between the server locations of exchanges i and j. For any subset $C \subseteq \mathcal{E}$, we define the diameter by

$$d(C) := \max_{i,j \in C} \ell(i, j)$$

A visual representation of what the distributed architecture looks like is given in the following figure:

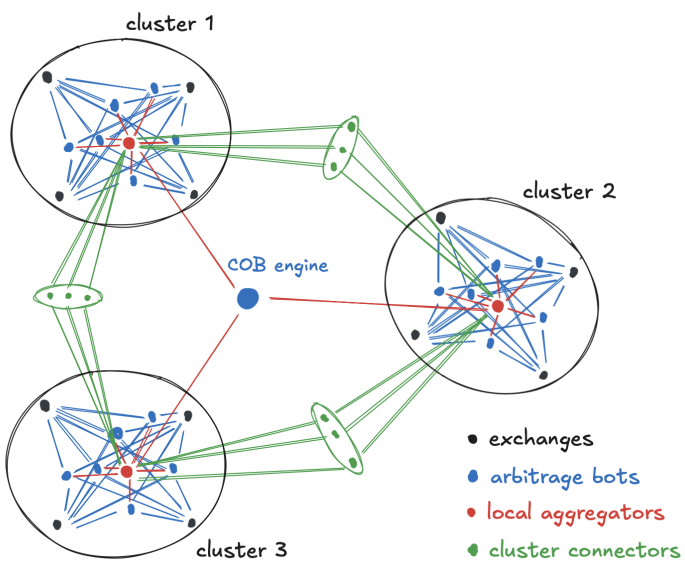

Figure 1: Oracle Architecture

The clustering is obtained by partitioning the set of exchanges into

$$\mathcal{E} = \bigsqcup_{k=1}^{K} C_k$$

such that the maximum cluster diameter is minimized subject to the hard constraint $d(C_k) \leq \tau_{\max}$ for a chosen threshold $\tau_{\max}$.

2.3 Arbitrage Agents and Local Aggregators

Within each cluster C_k, a potentially large number of independent arbitrage agents operate. Each agent being connected to all exchanges in C_k with very low latency (by construction). For any deviation between exchanges in C_k that exceeds a profitability threshold accounting for fees, slippage and adverse selection risk, the agent executes an arbitrage. The aggregate effect of these agents is to enforce convergence toward the no-arbitrage condition inside their assigned cluster. The security of the resulting price feed against manipulation increases with the total deployed capital and the number of independent participants.

For each cluster C_k, a small committee $\mathcal{A}_k$ of independent aggregator nodes is responsible for producing the local consolidated book. Each aggregator receives raw order book updates from all arbitrage agents in C_k and applies Algorithm (1), they use TEEs to ensure the Engine can trust their output. We will later explore the importance of local aggregators compared to having arbitrage bots communicate directly with the Engine.

2.4 Cluster Connectors

Cluster Connectors improve information percolation in the market, leading to faster convergence to f_t across different clusters. The concrete study of how much they improve convergence of prices in each cluster to f_t is left for later.

2.5 Engine Nodes

The Engine consists of a small replicated set $\mathcal{N}$ of n co-located deterministic nodes. Each Engine node receives the robustly aggregated cluster books, constructs the global bid and ask depth profiles $\mathbf{B}^{(t)}$ and $\mathbf{A}^{(t)}$, and executes Algorithm (1) to compute the current fair value f_t. The resulting fair value, together with a commitment to the input cluster books, is continuously signed and broadcast. Each node within this cluster operates using a TEE to ensure a trustless process, having $n > 1$ simply linearly scales the difficulty of a DoS attack.

3 Robustness

Above we've been focused on accuracy, which is great, but not the primary motivation for the existence of oracles. Indeed oracles are essentially used for their robustness properties, namely that they can not be manipulated without significant loss. Ideally, a great oracle should have the property that manipulating its price would be equivalent to manipulating the entire market, leaving no vulnerable cracks.

3.1 The two cracks in the wall

There are two attacks that are possible due to assumptions we have made above. The first assumption is that API outputs from exchanges are actionable, ie. you can really buy and sell at those prices. The second is a market manipulation technique. Both involve the same attack vector, the latter is harder to detect.

Assumption 1. *The oracle price f_t is intentionally delayed by a latency $\Delta t > 0$ before being published/updated. This delay allows the oracle to verify that the observed price is "good" (e.g., no ongoing manipulation or anomaly) before reporting it. We further assume that the diameter of any cluster diameter satisfies $\delta_k \ll \Delta t$, meaning agents in a cluster can exploit any temporary price deviation before the oracle updates its price.*

Theorem 3.1. *Assume (1). The cost $c^{(t)}(\epsilon)$ to skew the oracle price f_t by ϵ for a positive duration is upper bounded by*

$$C^{(t)}(\epsilon) = \sum_{j \in \mathcal{E}} \sum_{p_k \in [f_t - \epsilon, f_t]} (p_k - f_t) \cdot b_{j,k}^{(t)} \tag{4}$$

Sketch Proof. Suppose the attacker is on exchange $i \in \mathcal{E}$, the successful oracle price manipulation amounts to perform the following procedure :

1. Buys all bids in i whose prices range from $f_t - \epsilon$ to f_t
2. Places a huge sell order (ask) at price $f_t - \epsilon$ with depth $D^{(t)}(\epsilon)$
3. Maintain the fact that his ask position is the best ask throughout all exchanges' order books in $\mathcal{E}$ for time Δt, to ensure people don't fill the created arbitrage.

From this procedure, we see that, in order to fool Algorithm (1.2) into calculating a fair price of $f_t - \epsilon$, $D^{(t)}(\epsilon)$ must be higher than the total sum of bid volumes in $\mathcal{E}$. Furthermore, the above has to hold throughout all $[f_t - \epsilon, f_t]$ in order to support point 3. The depth needed to skew f_t by ϵ after no-arbitrage exhaustion should be at least :

$$D^{(t)}(\epsilon) \geq \sum_{j \in \mathcal{E}} \sum_{p_k \in [f_t - \epsilon, f_t]} b_{j,k}^{(t)} \tag{5}$$

Since this volume will be sold to the attacker at prices p_k, the cost function for the attack is :

$$\sum_{j \in \mathcal{E}} \sum_{p_k \in [f_t - \epsilon, f_t]} (p_k - f_t) \cdot b_{j,k}^{(t)}$$

Finally, point 3 shows that the attacker will incur an additional loss based on how much market buy volume will arrive during the interval Δt. □

Let us now attempt to quantify what the additional cost is to the above attack. Assume that for a given cluster k, we distribute a software package to N_k agents. Each player proves to the COB (if possible) that they are running the arbitrage bot with liquidity $q_{j,i}$, $j \in [N], i \in \mathcal{E}$. This means that at any time, agent i can sell or buy volume $q_{j,i}$ on exchange $i \in C_k$.

Definition 3.2. *A cluster C_k is said to be Q-secure at time t if the total arbitrage liquidity committed by the bots in the cluster satisfies*

$$Q_k^{(\Delta t)} \geq Q \tag{6}$$

where

$$Q_k^{(\Delta t)} := \sum_{j=1}^{N_k} \min\left(q_{j,i}^{(t)}, \sum_{m \in C_k \setminus \{i\}} q_{j,m}^{(t)} \right) \tag{7}$$

is the total volume that will be bought up by the DAN, with latency $\leq \Delta t$, in cluster k.

Remark 3.3. *The min in equation (7) arises from the following : if an agent's liquidity is imbalanced across exchanges in the cluster (e.g., high on i but low elsewhere), their effective arbitrage capacity is bottlenecked by the minimum of their liquidity on i (for buying the skewed asset) and their total liquidity on the other exchanges in $C_K \setminus \{i\}$ (for selling it back at the fair price).*

We now set ourselves in a Q-cluster. In this case we have that, when creating a sell order in $f_t - \epsilon$, and assuming there is sufficient liquidity on other CEXs of the cluster, the attacker is exposed to additional loss

$$\epsilon \cdot \min\left(D^{(t)}(\epsilon), Q_k^{(\Delta t)}(\epsilon) \right)$$

3.2 Robustness to market manipulation

Theorem 3.4. *Assume C_k is Q-secure. The attacker's net utility is upper bounded by*

$$U_i^{(t)}(\epsilon) \leq \epsilon \left(f(\text{TVS}) - \min\big(D^{(t)}(\epsilon), Q\big) \right) - c^{(t)}(\epsilon) \tag{8}$$

Proof. During Δt, the goal of the local arbitrage bots network is to snipe the attacker's quote in $f_t - \epsilon$ as much as possible, with total possible executed volume of $D^{(t)}(\epsilon)$. The utility function of the attacker, after having skewed the price, is

$$\epsilon \cdot \Big[\underbrace{f(\text{TVS})}_{\text{oracle failure}} - \underbrace{\min\left(D^{(t)}(\epsilon), Q_k^{(\Delta t)} \right)}_{\text{SAN quote-sniping}} \Big] \tag{9}$$

where f is a linear function. In turn this means that the more the oracle secures, the more volume $Q_k^{(\Delta t)}$ is needed to counter act the potential attack incentive. To sustain the skew until the oracle updates, the attacker must post ask depth at least $D^{(t)}(\epsilon)$ at price $f_t - \epsilon$ on the targeted exchange and incur sweeping cost $c^{(t)}(\epsilon)$. By Assumption (1) ($\delta_k \ll \Delta t$) and the definition of $Q_k^{(\Delta t)}$ (7), the DAN detects the arbitrage and executes up to $Q_k^{(\Delta t)} \geq Q$ volume against the attacker's quote within Δt. The sniped volume is therefore exactly $\min\big(D^{(t)}(\epsilon), Q\big)$, yielding a loss of $\epsilon \cdot \min\big(D^{(t)}(\epsilon), Q\big)$. The stated utility upper bound follows directly.

□